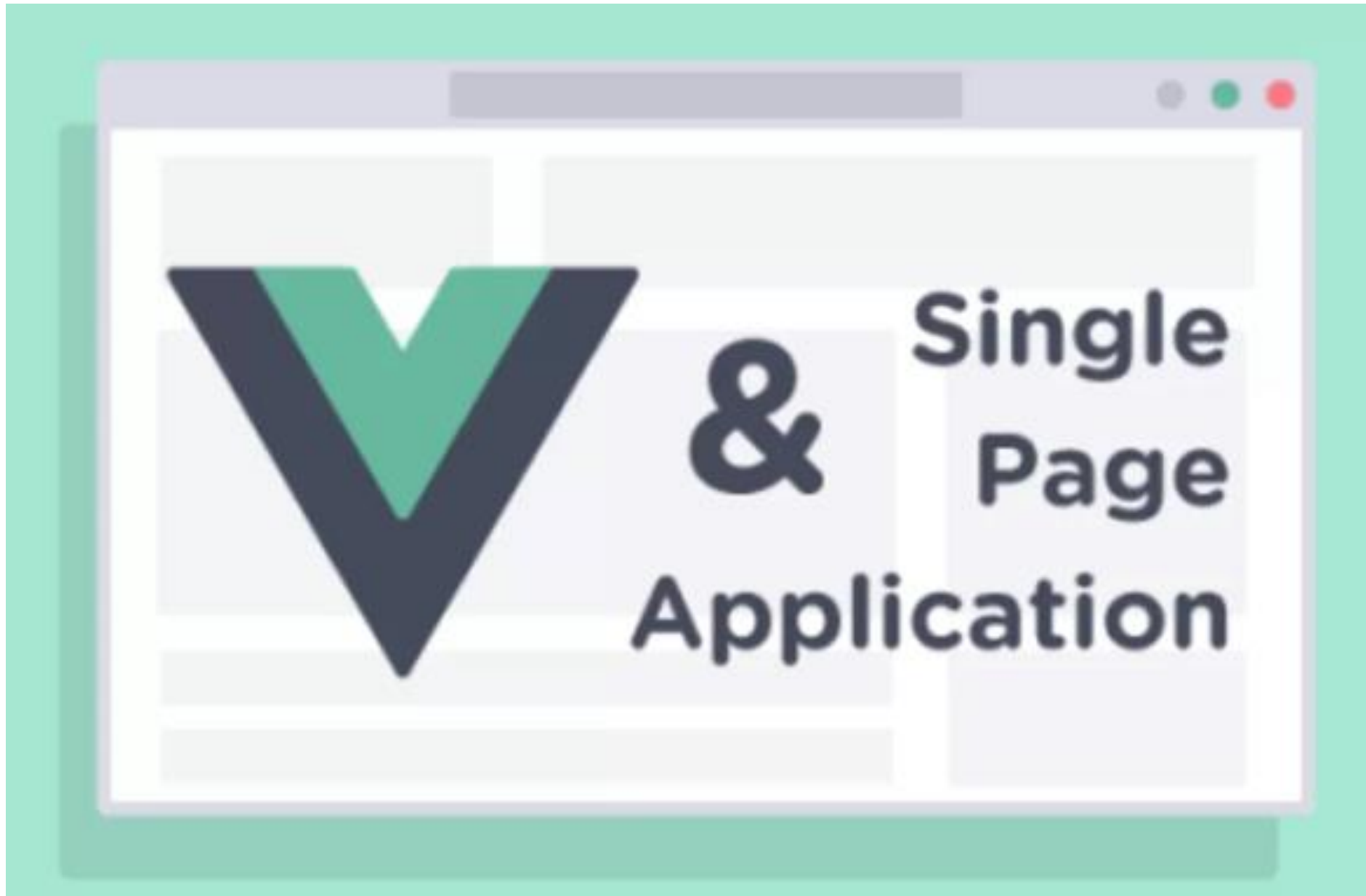




It's Your Life

with







컴포넌트



<https://ko.vuejs.org/guide/essentials/component-basics.html#components-basics>

컴포넌트 기초

컴포넌트를 사용하면 UI를 독립적이고 재사용 가능한 일부분으로 분할하고 각 부분을 개별적으로 다룰 수 있습니다. 따라서 앱이 중첩된 컴포넌트의 트리로 구성되는 것은 일반적입니다:





컴포넌트 왜? 쓰나요!?

- 사실 컴포넌트도 JS 의 함수, 그리고 Vue 의 메서드와 마찬가지로 중복된 코드를 쉽게 재사용하기 위해서 사용합니다~!

컴포넌트를 사용하면 UI를 독립적이고 재사용 가능한 일부분으로 분할하고 각 부분을 개별적으로 다룰 수 있습니다. 따라서 앱이 중첩된 컴포넌트의 트리로 구성되는 것은 일반적입니다:



일단 코드를 봅시다!



```
<script setup></script>
<template>
  <div class="todo">
    <h1>오늘의 할일</h1>
    <div class="checklist">
      <h2>오늘 해야할 일 2개</h2>
      <h2>오늘 완료한 일 0개</h2>
    </div>
    <hr />
    <h2>Vue 공부하기</h2>
    <hr />
    <h2>배열 함수 끝장내기</h2>
    <hr />
  </div>
</template>
<style scoped>
.todo { text-align: center; }
</style>
```

오늘의 할일

오늘 해야할 일 2개

오늘 완료한 일 0개

Vue 공부하기

배열 함수 끝장내기

```
<script setup></script>
<template>
  <div class="todo">
    <h1>오늘의 할일</h1>
    <div class="checklist">
      <h2>오늘 해야할 일 2개</h2>
      <h2>오늘 완료한 일 0개</h2>
    </div>
    <hr />
    <h2>Vue 공부하기</h2>
    <hr />
    <h2>배열 함수 끝장내기</h2>
    <hr />
  </div>
</template>
<style scoped>
.todo { text-align: center; }
</style>
```

갑자기 요 checklist div 를
최하단에도 추가하고 싶다면!?





```
<script setup></script>
<template>
  <div class="todo">
    <h1>오늘의 할일</h1>
    <div class="checkboxlist">
      <h2>오늘 해야할 일 2개</h2>
      <h2>오늘 완료한 일 0개</h2>
    </div>
    <hr />
    <h2>Vue 공부하기</h2>
    <hr />
    <h2>배열 함수 끝장내기</h2>
    <hr />
    <div class="checkboxlist">
      <h2>오늘 해야할 일 2개</h2>
      <h2>오늘 완료한 일 0개</h2>
    </div>
  </div>
</template>
<style scoped>
.todo { text-align: center; }
</style>
```

HTML에서는 어쩔 수 없이
HTML 코드를 그대로 복사해서
써야만 했습니다!

그런데 말입니다.....



- 그런데 저런 checklist 를 약 1000000 개의 페이지에서 다 쓴다면!?
- 그리고 checklist 에 갑자기 내용이 추가 되어야 한다면?





두둥등잠



컴포넌트



반복되는 HTML 요소들을 컴포넌트로!



```
<template lang="">
  <div class="checklist">
    <h2>오늘 해야할 일 2개</h2>
    <h2>오늘 완료한 일 0개</h2>
  </div>
</template>

<script>
export default {
  name: 'CheckList',
};
</script>

<style lang=""></style>
```

```
<script setup>
import CheckList from
'./components/CheckList.vue';
</script>

<template>
  <div class="todo">
    <h1>오늘의 할일</h1>
    <CheckList />
    <hr />
    <h2>Vue 공부하기</h2>
    <hr />
    <h2>배열 함수 끝장내기</h2>
    <hr />
    <CheckList />
  </div>
</template>
```



오늘의 할일

오늘 해야할 일 2개

오늘 완료한 일 0개

Vue 공부하기

배열 함수 끝장내기

오늘 해야할 일 2개

오늘 완료한 일 0개







```
<style>
.todo {
  text-align: center;
}
</style>
<template>
  <div class="todo">
    <h1>오늘의 할일</h1>
    <div class="checklist">
      <h2>오늘 해야할 일 2개</h2>
      <h2>오늘 완료한 일 0개</h2>
    </div>
    <hr />
    <h2>Vue 공부하기</h2>
    <hr />
    <h2>배열 함수 끝장내기</h2>
    <hr />
    <div class="checklist">
      <h2>오늘 해야할 일 2개</h2>
      <h2>오늘 완료한 일 0개</h2>
    </div>
  </div>
</template>
```

이 친구들도 뭔가
반복적으로 쓰이고 있군요!?

그럼 이 친구들도
컴포넌트로 처리할 수 있지 않을까요?







얘네는 서로 내용이 다른데요!?

서로 내용이 다르면
다른 컴포넌트로 만들어야 하는거
아닌가요!?

```
<style>
.todo {
  text-align: center;
}
</style>
<template>
  <div class="todo">
    <h1>오늘의 할일</h1>
    <div class="checklist">
      <h2>오늘 해야할 일 2개</h2>
      <h2>오늘 완료한 일 0개</h2>
    </div>
    <hr />
    <h2>Vue 공부하기</h2>
    <hr />
    <h2>배열 함수 끝장내기</h2>
    <hr />
    <div class="checklist">
      <h2>오늘 해야할 일 2개</h2>
      <h2>오늘 완료한 일 0개</h2>
    </div>
  </div>
</template>
```



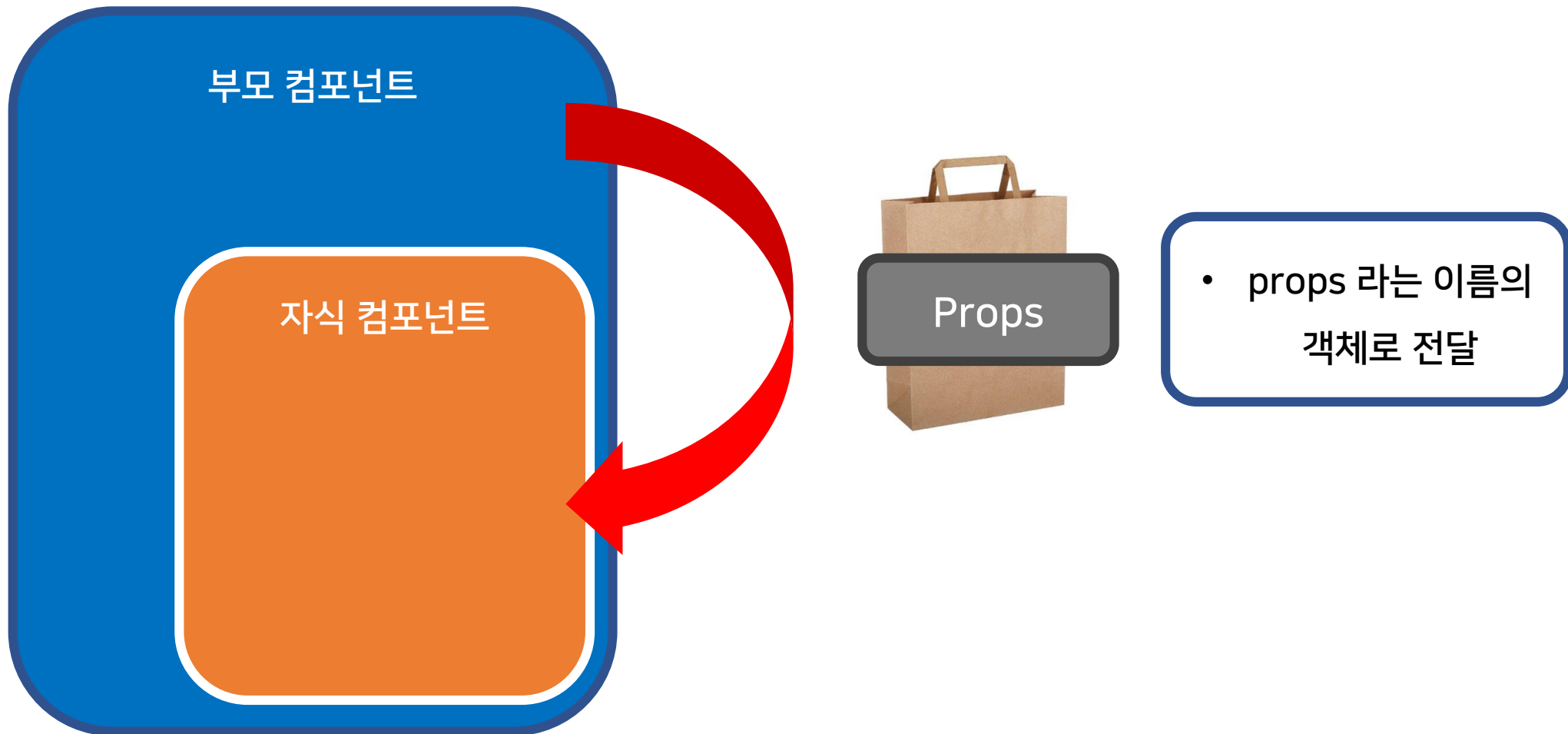


Props

Props?



- 부모 컴포넌트에서 자식 컴포넌트에 원하는 데이터를 보내는 방법입니다!



가수 이소위
어머니의 마음

Parent
Component

Child
Component

Props

♪ 어려선 안고 업고 올려주시고



```
<template>
  <div>
    <h2>{{ todo }}</h2>
    <hr />
  </div>
</template>
```

```
<script>
export default {
  name: 'TodoProps',
  props: ['todo'],
};
</script>
```

선언한 props 를
data 를 사용하는 것 처럼 바로 사용!

부모로부터 받아올 props 를
전부 배열에 넣어서 선언!



```
<script setup>
import CheckList from
'./components/CheckList.vue';
import TodoProps from
'./components/TodoProps.vue';
</script>

<template>
  <div class="todo">
    <h1>오늘의 할일</h1>
    <CheckList />
    <hr />
    <TodoProps todo="Vue 공부하기" />
    <TodoProps todo="배열 함수 끝장내기" />
    <CheckList />
  </div>
</template>
```

```
<template>
  <div>
    <h2>{{ todo }}</h2>
    <hr />
  </div>
</template>

<script>
export default {
  name: 'TodoProps',
  props: ['todo'],
};
</script>
```



```
<template>
  <div>
    <h1>부모 컴포넌트</h1>
    <!-- 문자 데이터 전달 -->
    <ChildComponent msg="문자열 입니다!" />
    <!-- 숫자 데이터 전달 -->
    <ChildComponent :count="5" />
    <!-- 객체 데이터 전달 -->
    <ChildComponent :user="{ name: 'John', age: 30 }" />
    <!-- 배열 데이터 전달 -->
    <ChildComponent :items="[1, 2, 3, 4, 5]" />
  </div>
</template>
```

문자열은 일반 속성 처럼
props이름="내용" 으로 보냅니다

문자열을 제외한 값들은
따옴표(") 내부의 의미가
JS적 의미로 전달이 되어야 하므로
반드시 v-bind 로 전달을 해야만 합니다!



오늘의 할일

오늘 해야할 일 2개

오늘 완료한 일 0개

Vue 공부하기

배열 함수 끝장내기

오늘 해야할 일 2개

오늘 완료한 일 0개







Props 의 다양한 활용

나는 천재다

Props

능히 할 수 있다



Props 는
데이터 뿐만 아니라
다양한 것들을 전달 가능합니다!

나는
할 수 없다



능이버섯이다



```
<template>
  <button @click="parentEvent">자식 버튼</button>
</template>
<script>
export default {
  name: 'EventProps',
  props: ['parentEvent'],
};
</script>
<style lang=""></style>
```

부모로 부터 받아온 이벤트를
버튼에 v-on:click 에 연결!

Props 로 Event 를 받기 위해서
왼쪽 처럼 선언을 해봅시다!



```
<script setup>
import CheckList from './components/CheckList.vue';
import TodoProps from './components/TodoProps.vue';
import EventProps from './components/EventProps.vue';
```

```
const sendingEvent = () => {
  alert('부모 컴포넌트님이 말씀 하셨습니다~ 비상!!!');
};
</script>
```

Vue3 문법을 활용하여
간단하게 이벤트를 정의!

```
<template>
  <div class="todo">
    <h1>오늘의 할일</h1>
    <CheckList />
    <hr />
    <TodoProps todo="Vue 공부하기" />
    <TodoProps todo="배열 함수 끝장 내기" />
    <CheckList />
    <EventProps :parentEvent="sendingEvent" />
  </div>
</template>
```

정의한 이벤트를
EventProps 컴포넌트에 전달!



EventProps 컴포넌트에 있는
자식 버튼을 클릭하면 어떻게 될까요?

오늘의 할일

오늘 해야할 일 2개

오늘 완료한 일 0개

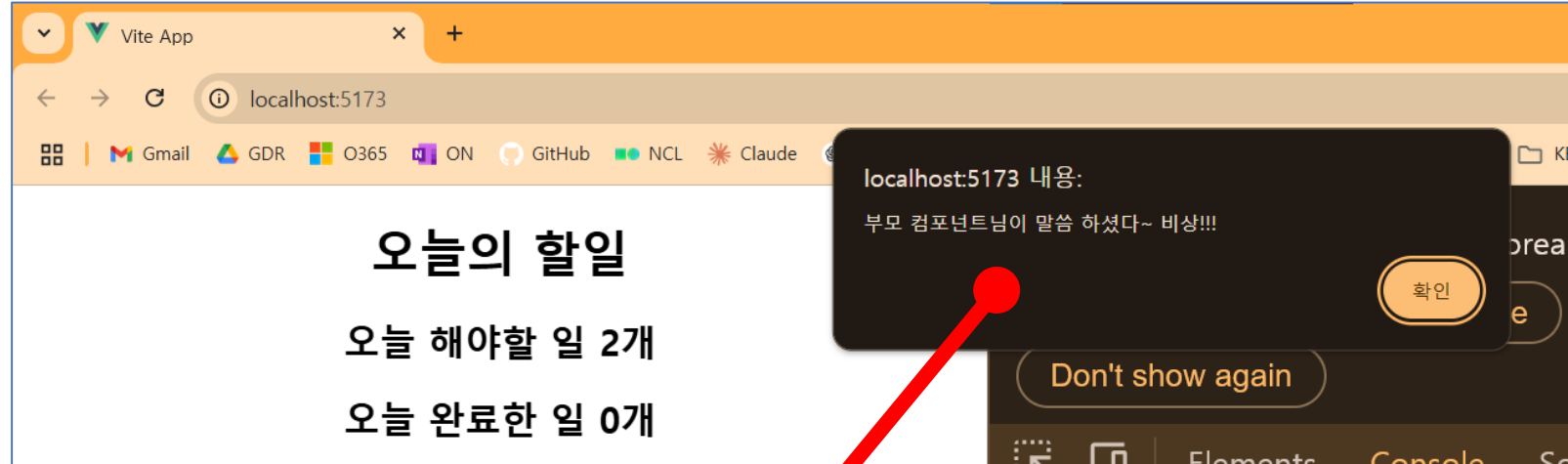
Vue 공부하기

배열 함수 끝장 내기

오늘 해야할 일 2개

오늘 완료한 일 0개

자식 버튼



부모로 받은 이벤트가 실행!



유동근 레고설

인간을 사랑한 레고



실습, 컴포넌트 및 props



- 부모 App 컴포넌트로 부터 각각 siteName, siteUrl, visited 라는 props 를 받아서 아래와 같은 내용을 출력하는 <PropsList /> 컴포넌트를 작성해 주세요
- 아래의 부모 App 컴포넌트의 코드와 결과물 스크린 샷을 참고해서 컴포넌트를 작성해 주세요!
- 추가, props 를 하나의 siteInfo 라는 객체로 넘겨서 위와 같은 동작을 하는 <PropsList2 /> 컴포넌트도 만들어 주세요



```
<template>
  <div>
    <PropsList siteName="네이버" siteUrl="http://naver.com" visited="방문" />
    <PropsList siteName="다음" siteUrl="http://daum.net" visited="미방문" />
    <PropsList2 :siteInfo="siteData" />
  </div>
</template>

<script>
export default {
  name: 'App',
  data() {
    return {
      siteData: {
        siteName: '페이스북',
        siteUrl: 'http://facebook.com',
        visited: '방문',
      },
    };
  },
};
```

네이버

[네이버 링크](#)

방문 여부 : 방문

다음

[다음 링크](#)

방문 여부 : 미방문

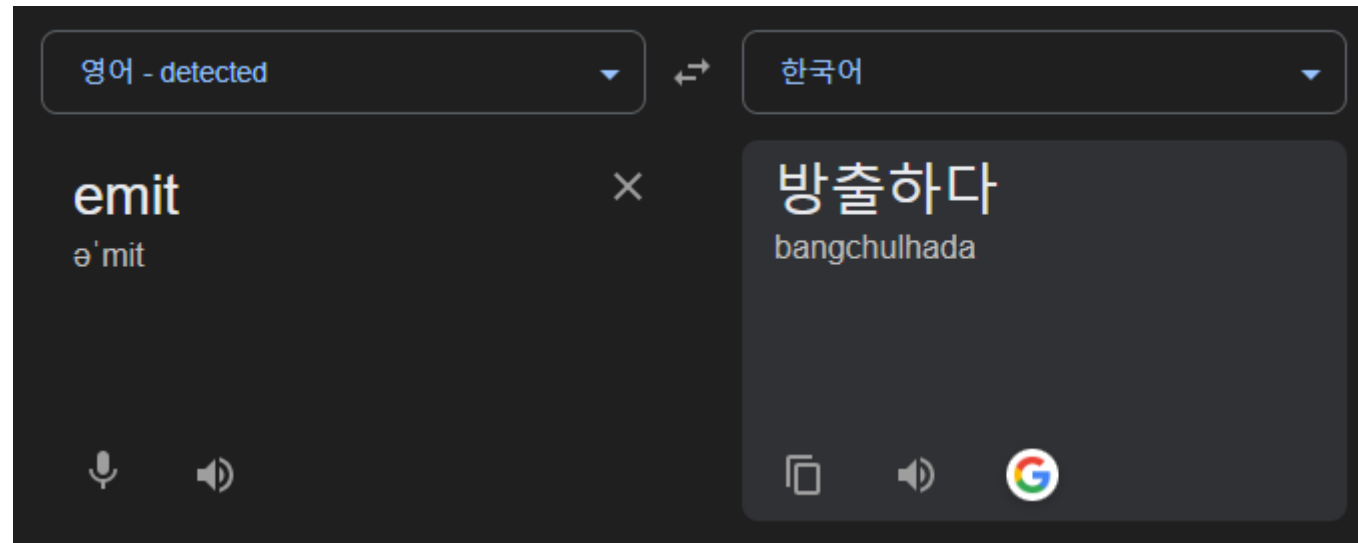
페이스북

[페이스북 링크](#)

방문 여부 : 방문



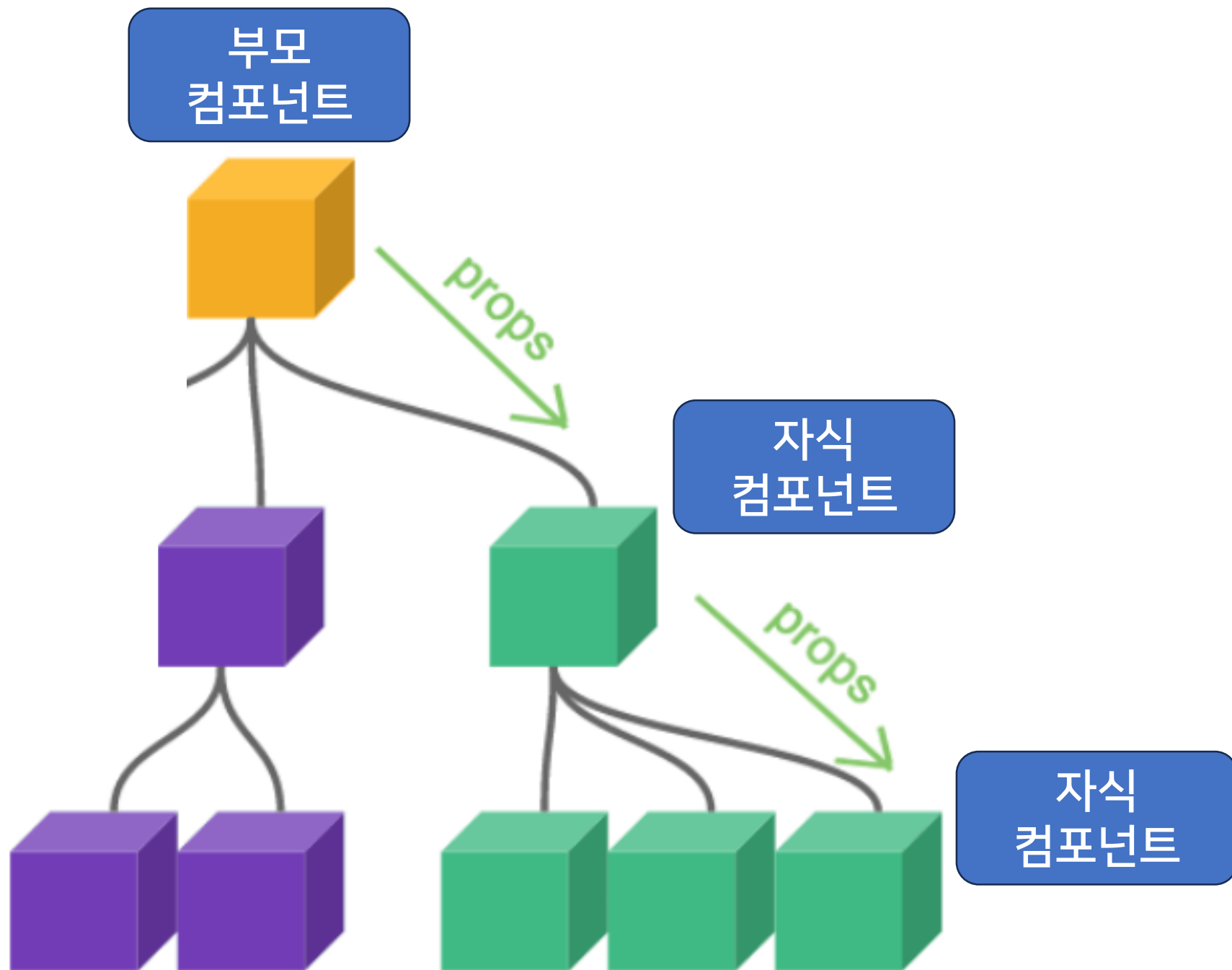
Emit





부모
컴포넌트

자식
컴포넌트



부모
컴포넌트



자식
컴포넌트

자식
컴포넌트



지미 🌹

@LOV3_OR_NOT



교수님...
위선도 선이고
악법도 법이면

오답도 답 아닌가요?!???!!



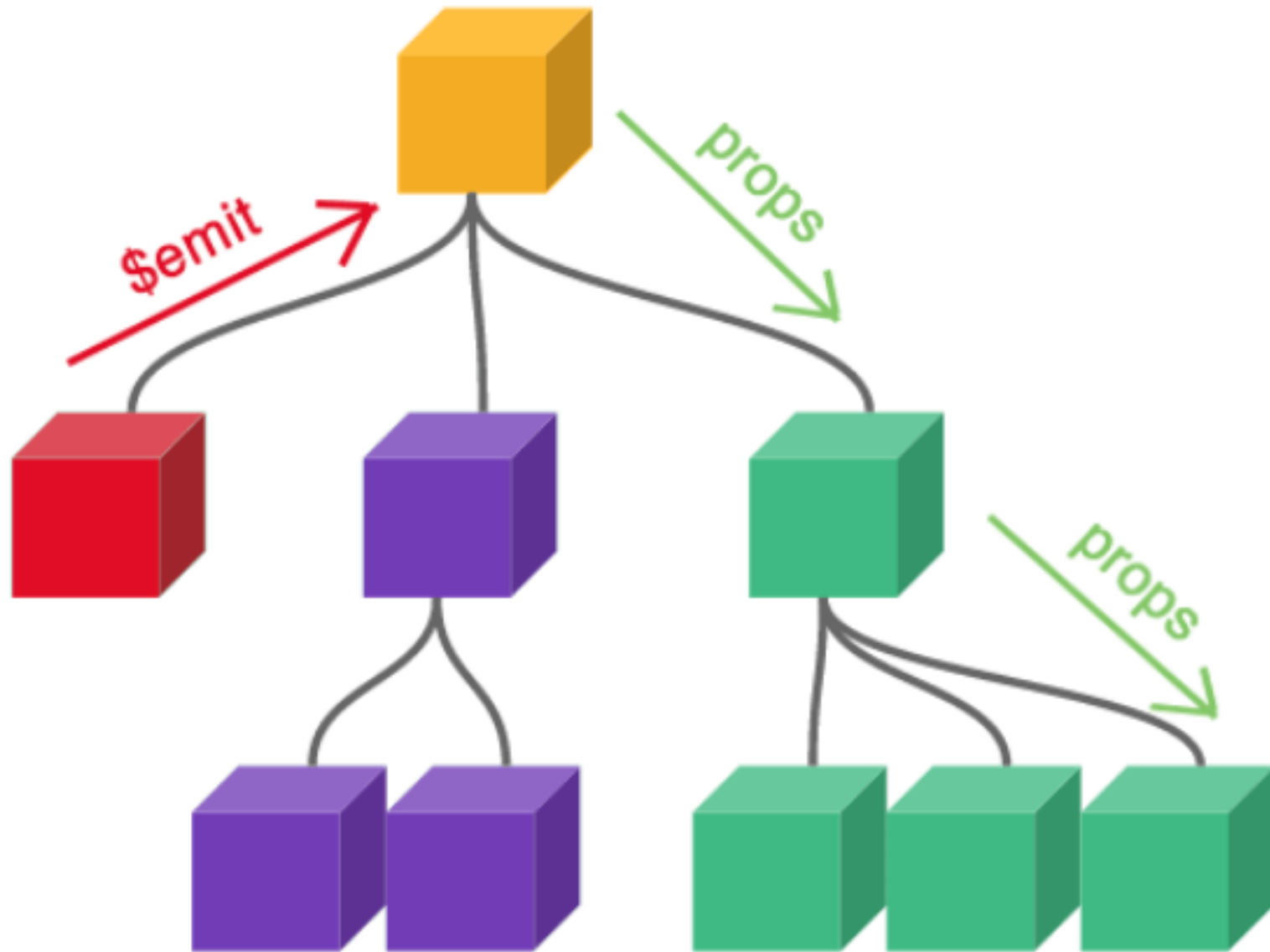


지미🌸

@LOV3_OR_NOT

교수님...
위선도 선이고
악법도 법이면

오답도 답 아닌가요?!???!!







Emit 이라 쓰고 주객전도라 읽는다!

- emit 은 부모가 자식에게 무언가를 시키는게 아니라, 반대로 자식이 부모에게 일을 시키는 일명 불속성 효자 개념입니다!
- 부모 컴포넌트의 데이터를 자식에게 보낼 때는!?
- 반면, 자식 컴포넌트에서 작성한 데이터를 부모 컴포넌트에 보낼 때는 약간 복잡한 과정을 거쳐야 하는데 그 일을 해주는 것이 emit 입니다!





Hyunmin Jang ▶ 아지매들

3시간 전 • 



엄마 판다는 새끼가 있네요



상황을 가정해 봅시다!



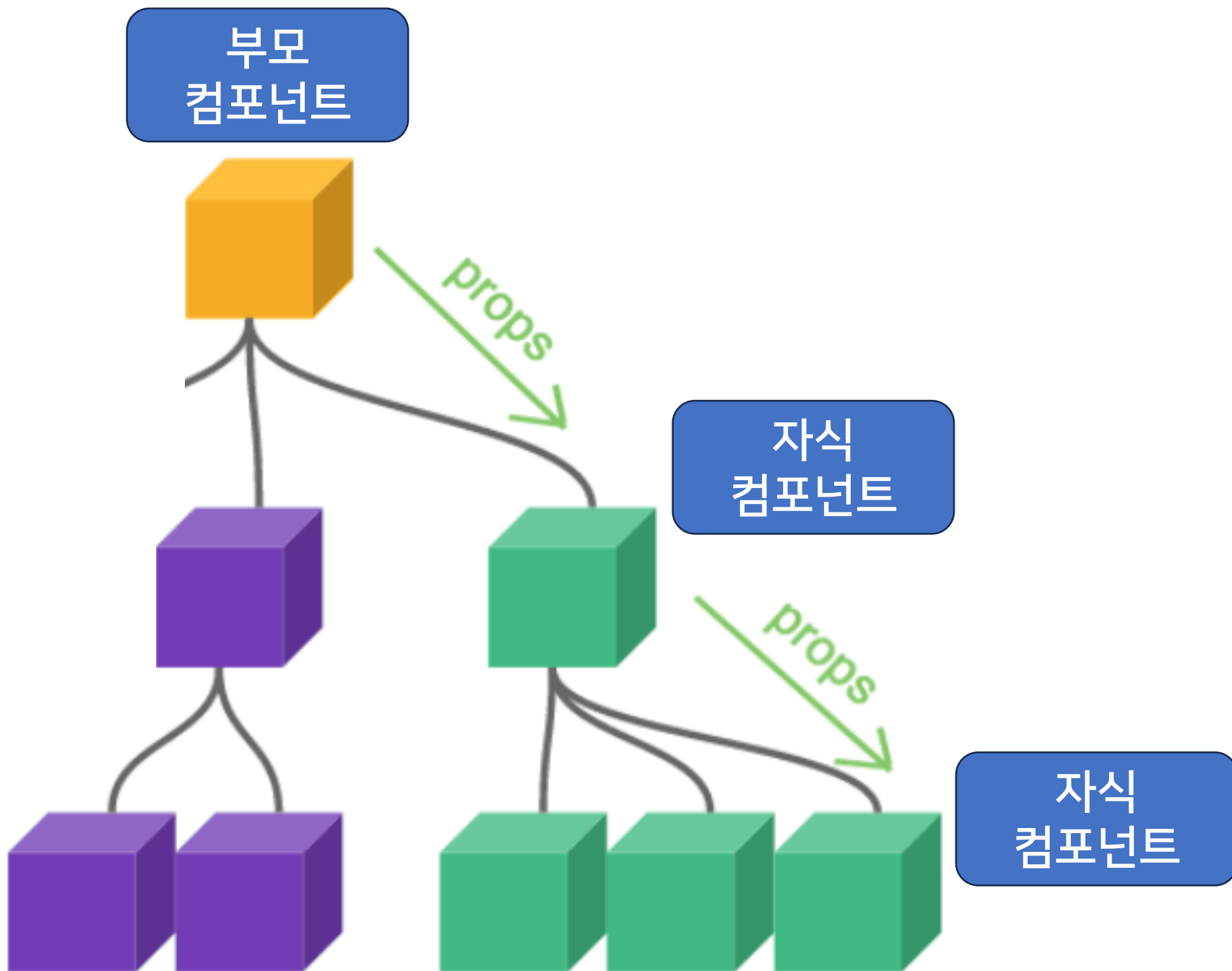
```
<script setup>
import SendNameComponent from
  './components/SendNameComponent.vue';
</script>
<template>
  <div>
    <h1>너의 이름은 {{ name }}</h1>
    <SendNameComponent />
  </div>
</template>
<script>
export default {
  name: 'App',
  data() {
    return {
      name: '',
    };
  },
};
</script>
```

App

```
<template>
  <div>
    <input type="text" v-model.trim="name" />
    <button>이름 전달!</button>
  </div>
</template>

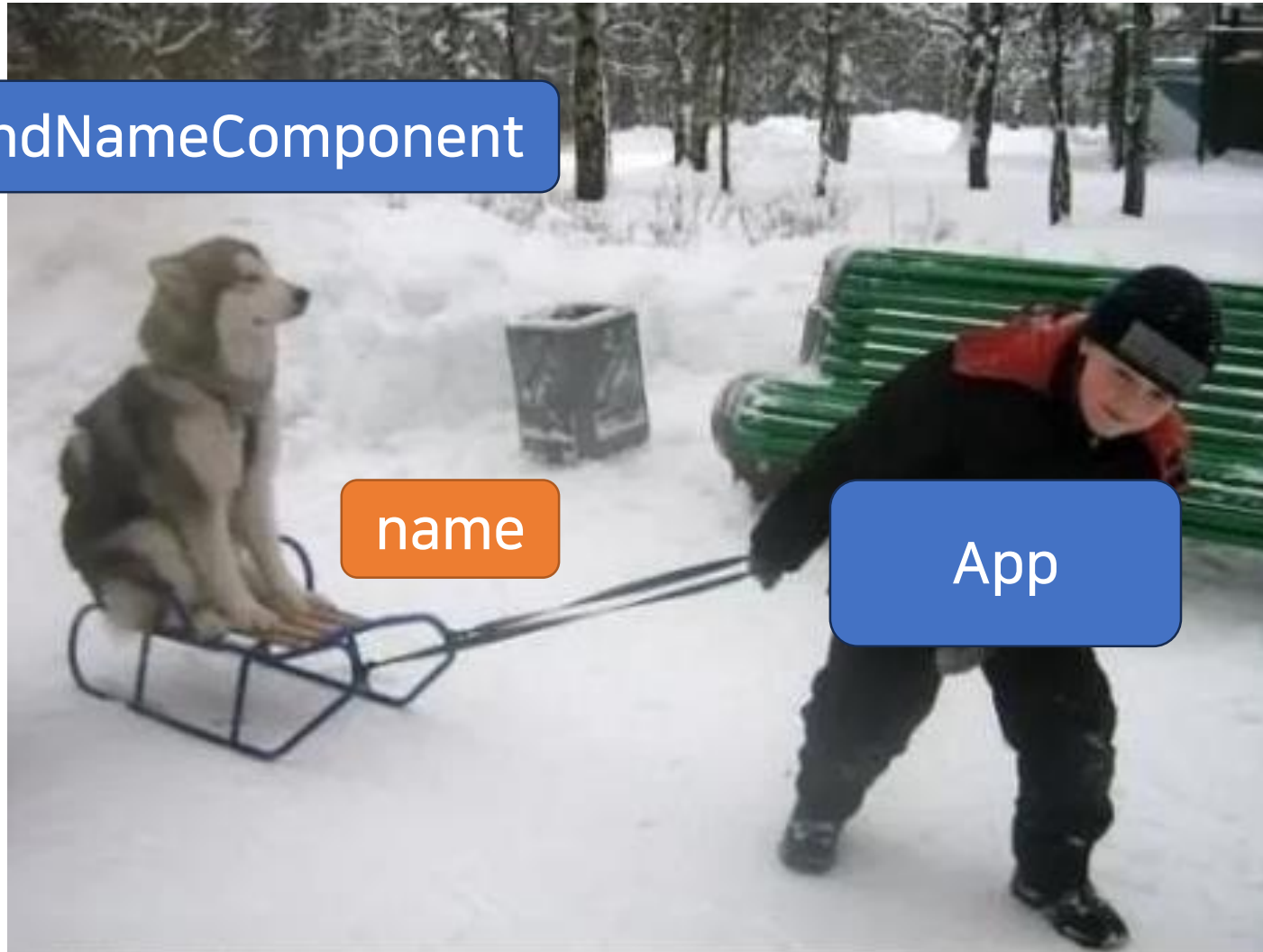
<script>
export default {
  name: 'SendNameComponent',
  data() {
    return {
      name: '',
    };
  },
};
</script>
```

SendNameComponent





SendNameComponent



name

App

상황을 가정해 봅시다!



```
<script setup>
import SendNameComponent from
  './components/SendNameComponent.vue';
</script>
<template>
  <div>
    <h1>너의 이름은 {{ name }}</h1>
    <SendNameComponent />
  </div>
</template>
<script>
export default {
  name: 'App',
  data() {
    return {
      name: '',
    };
  },
};
</script>
```

App

```
<template>
  <div>
    <input type="text" v-model.trim="name" />
    <button>이름 전달!</button>
  </div>
</template>

<script>
export default {
  name: 'SendNameComponent',
  data() {
    return {
      name: '',
    };
  },
};
</script>
```

SendNameComponent



지미 🌹

@LOV3_OR_N0T



교수님...
위선도 선이고
악법도 법이면

오답도 답 아닌가요?!???!!



SendNameComponent 자식 컴포넌트

```
<template>
  <div>
    <input type="text" v-model.trim="name" />
    <button @click="$emit('nameChanged', { name })">이름 전달!</button>
  </div>
</template>
```

```
<script>
export default {
  name: 'SendNameComponet',
  data() {
    return {
      name: '',
    };
  },
};
</script>
```

click 이벤트에 emit 을 걸어주기
+ 부모에서 해당 emit 을 받아줄
이벤트 이름 걸어주기

부모로 보내고 싶은 데이터를
객체에 넣어서 전달



```
<template>
  <div>
    <h1>너의 이름은 {{ name }}</h1>
    <SendNameComponent @nameChanged="nameChangeHandler" />
  </div>
</template>

<script>
export default {
  name: 'App',
  data() {
    return {
      name: '',
    };
  },
  methods: {
    nameChangeHandler(e) {
      this.name = e.name;
    },
  },
};
</script>
```

emit 에 붙여준 이름의 이벤트를
v-on 디렉티브로 연결

emit 이 실행되면 해당 이벤트가 호출

emit 에 의해 이벤트가 발생하면
이벤트를 처리해줄 메서드 연결

emit 에서 객체 { } 에 담아서 보냈던
데이터를 e 로 받아서 사용!



너의 이름은

이름 전달!

너의 이름은 루피

루피

이름 전달!



03년생





자식
컴포넌트

```
<button @click="$emit( ' 호출할이벤트명 ' , { 전달할데이터1, 전달할데이터2 } )"></button>
```

부모 컴포넌트에서
호출 할 이벤트명

전달 할 데이터를
하나의 객체로 묶어서 전달

```
<SendNameComponent @호출한이벤트명="이벤트를처리할메서드" />
```

부모
컴포넌트

```
methods: {  
  이벤트를처리할메서드(e) {  
    this.name = e.전달할데이터1;  
  },  
},
```

실습, Emit 사용하기



- 자식 컴포넌트에서 버튼을 클릭하면 자식 컴포넌트의 input 에서 입력 받은 색상을 부모 컴포넌트 div 요소의 배경색을 바꾸는 App 을 만들어 주세요
- 자식 컴포넌트의 이름은 <ColorEmit /> 입니다

```
<template>
  <div :style="{ backgroundColor: color }">
    <h1>저의 색상을 정해주세요!!</h1>
    <ColorEmit />
  </div>
</template>
```

```
<script>
export default {
  name: 'App',
  data() {
    return {
      color: '',
    };
  },
};
</script>
```

App

```
<template>
  <div>
    <input type="text" v-model.trim="color" />
    <button>색상 전달</button>
  </div>
</template>
```

```
<script>
export default {
  name: 'ColorEmit',
  data() {
    return {
      color: '',
    };
  },
};
</script>
```

ColorEmit